

Canal de apps de terceros — login por request + código temporal (R18, 2026-06-17)

Esto **NO** es para el portal web. El portal sigue con JWT Bearer + refresh (sin cambios). Esta sección documenta un canal **adicional y paralelo** pensado para **integradores/apps externas** que pegan directo a Koape. Se documenta acá para tener el contrato completo en un solo lugar.

Qué es

En vez de loguearse y mantener un token de 15 min, el integrador manda **en CADA request** tres headers y se valida todo en el acto ("conexión de un solo uso", sin token persistente):

Header	Valor
X-Auth-User	usuario (el mismo del login JWT)
X-Auth-Password	su contraseña
X-Auth-Codigo	código de acceso temporal vigente (rota cada 1–5 h)

Conviven los tres esquemas: si llega `Authorization: Bearer` → JWT (web); si llegan los `X-Auth-*` → este canal; si llega `X-API-Key` → API key (existe pero ya no se usa). No se pisan.

El código temporal y su rotación

- El **primer código** lo genera un `maestro/admin` (ver endpoint de provisión abajo). Se muestra **una sola vez**.
- Rota **automáticamente cada 1–5 h** (perezoso: rota en el primer request pasado el plazo, no hay cron).
- Cuando rota, la respuesta del request que usó el código viejo trae el **nuevo**:
 - Header `X-Auth-Codigo-Nuevo: <nuevo>`
 - Y, si la respuesta es JSON, campo `"nuevo_codigo": "<nuevo>"`
- El cliente **debe leer ese header/campo en cada respuesta y actualizar su código almacenado**. El código viejo sigue siendo válido hasta que el cliente use el nuevo → si se pierde una respuesta, no hay lockout (se re-entrega).

Provisión del primer código

POST `/api/v1/auth/usuarios/{id}/credencial-temporal/regenerar/` — autenticado con **JWT** de un `maestro/admin` (un admin solo sobre usuarios de sus empresas).

Response 201 (una sola vez):

```
{
  "usuario": "app_externa",
  "codigo_acceso": "R4LX4zVK0Tbk",
  "detalle": "Guardá este código: no se vuelve a mostrar. Enviálo en cada request por el header X-Auth-Codigo..."
}
```

Lamarlo de nuevo **rota** el código (resetea: el anterior deja de servir).

Quitar el acceso por este canal (no afecta el user/password ni el acceso web/JWT del usuario):

- POST /api/v1/auth/usuarios/{id}/credencial-temporal/desactivar/ → 204. Pone **activo=false**: el código deja de autenticar de inmediato y **no se genera/rota nada** mientras está inactivo. Para volver a habilitar el canal se usa **regenerar**, que emite un **código nuevo** (no se recupera el anterior). 404 si el usuario no tenía credencial activa.

Emitir por este canal (respuesta **SÍNCRONA** con el primer estado de SIFEN)

A diferencia del canal web (async → **pendiente**), por este canal la emisión **bloquea hasta 10 s** y devuelve el **primer estado real de SIFEN**:

```
curl -X POST https://api.facturacionkoape.com/api/v1/documentos/emitir/ \
-H 'Content-Type: application/json' \
-H 'X-Auth-User: app_externa' \
-H 'X-Auth-Password: CLAVE DEL USUARIO' \
-H 'X-Auth-Codigo: R4LX4zVK0Tbk' \
-d '{
  "header": { "empresa_ruc": "80168651", "empresa_dv": "2" },
  "body": {
    "tipo_documento": "FE",
    "documento": {
      "cabecera": {
        "establecimiento": "001", "punto_expedicion": "001", "numero_documento": "0000001", "fecha_emision": "2026-06-17T10:00:00", "moneda": "PYG", "condicion_venta": "contado" },
        "cliente": { "ruc": "80012345", "dv": "6", "nombre": "Distribuidora del Este S.A." },
        "items": [ { "descripcion": "Producto A", "cantidad": "2.0000", "precio_unitario": "150000.0000", "iva_tipo": "1" } ],
        "pagos": [ { "tipo": "1", "monto": "300000.0000" } ],
        "incluir_kude_base64": true
      }
    }
  }
}'
```

El cuerpo del documento (cabecera/cliente/items/pagos/credito) es **idéntico** al de la sección [POST /api/v1/documentos/emitir/](#) más abajo (mismos campos, mismas reglas, mismos tipos FE/AFE/NCE/NDE/NRE). Lo único distinto es la **autenticación por headers** y la **respuesta**.

Response 201:

```
{
  "status": "ok",
  "id": 123,
  "cdc": "01801686510010010000000012026061719811615252",
  "estado": "aprobado",
  "codigo_sifen": "0260",
  "mensaje_sifen": "Autorizacion satisfactoria",
  "kude_base64": "JVBERi0xLjQ..."
}
```



Interpretación de estado:

estado	Significado	Acción
aprobado / aprobado_con_observacion	SIFEN autorizó	Listo. kude_base64 presente
rechazado	SIFEN rechazó	Ver mensaje_sifen; corregir
pendiente	SIFEN no respondió en 10 s (fallback)	Consultar luego por CDC e

- **incluir_kude_base64: true** (en documento): devuelve el KuDE (PDF) en base64 **solo si quedó aprobado**. No se almacena ni se envía a SIFEN. Si no quedó aprobado → "kude_base64": null.
- Los demás endpoints (consultar estado, KuDE, listados, eventos) aceptan los mismos headers x-Auth-* (los GET no llevan body; las credenciales van siempre en headers).

Por qué un JWT directo se bloquea con "Usá el portal web"

El endpoint de emisión corre `bloqueo_emision_api_directa` (`core/permissions.py`), que **no** distingue "token vs api key" sino **el canal de origen**, en orden:

1. Canal código temporal (x-Auth-*) → permite.
2. API key (`ApiCliente`) → permite.
3. **JWT + header** `x-Portal-Secret == PORTAL_SHARED_SECRET` → permite (es el portal).
4. Usuario `maestro` → permite.
5. Usuario con 'api' en `tipos_facturacion` → permite.
6. Si no → 403 **"Tu usuario no puede emitir al endpoint directo. Usá el portal web."**

El portal web funciona porque el **proxy de ui_web** **agrega** `x-Portal-Secret` al reenviar (regla 3). Es la misma vista y el mismo endpoint; lo que cambia es la prueba de origen. Una app externa demuestra su canal con la **API key**, con el **código temporal**, o teniendo el permiso 'api' en el usuario. El `x-Portal-Secret` es **exclusivo del portal** y no debe compartirse con integradores.

Para permitir que un usuario emita por JWT directo (sin portal), un `maestro/admin` le agrega el canal: `PATCH /api/v1/auth/usuarios/{id}/` CON `{"tipos_facturacion": ["api"]}`.

Seguridad / notas

- Validar la contraseña en cada request implica hashing por request (costo CPU): es el precio del "un solo uso". **Throttle dedicado** `temp_credential` (env `THROTTLE_TEMP_CREDENTIAL`, default **300/min por IP**), evaluado **antes** del hashing → un exceso devuelve 429 sin gastar CPU. Además siguen aplicando `user` (300/min) y, en emisión, `emision` (120/min). Subir el límite si el integrador hace alto volumen legítimo.

- La auditoría enmascara `codigo_acceso`, `nuevo_codigo` y `kude_base64` en los logs.
- **Bootstrap sin endpoint/shell** (p.ej. en prod): `python manage.py crear_credencial_temporal --username <user>` (imprime el código una vez; `--rotar` fuerza uno nuevo).
- Requiere `migrate` (`usuarios/0007`) + rebuild de `web`.

Roles y permisos

Los roles se almacenan en minúscula en la base de datos (`base`, `admin`, `maestro`). El portal puede mostrarlos en mayúsculas por legibilidad.

Rol	Acceso
<code>maestro</code>	Superadministrador. Acceso implícito a todas las empresas activas. Único que puede eliminar (soft delete) d
<code>admin</code>	Acceso a las empresas a las que está asignado vía N:N. Puede emitir, ver historial, gestionar usuarios <code>base</code> y empresas. No puede crear <code>maestro</code> ni asignar empresas a las que él mismo no tiene acceso.
<code>base</code>	Solo puede emitir documentos en las empresas asignadas vía N:N. No accede a templates ni gestión de usu

Vinculación usuario ↔ empresa

La relación es **N:N** mediante la tabla `usuarios_empresas`. El `maestro` no requiere filas: el helper `empresas_accesibles(user)` le devuelve todas las empresas activas implícitamente.

Endpoint	Descripción
<code>POST /api/v1/auth/usuarios/{id}/asignar-empresa/</code>	Vincula al usuario una empresa adicional. Body: <code>{"emp</code>
<code>POST /api/v1/auth/usuarios/{id}/desasignar-empresa/</code>	Elimina la asignación. Body: <code>{"empresa": <id>}</code> .

Reglas:

- `maestro` no necesita asignaciones (400 con mensaje claro si se intenta asignarle una empresa).
- `admin` solo puede asignar empresas a las que él mismo tiene acceso.
- El FK legacy `Usuario.empresa` fue eliminado en Fase 7. Las cuentas técnicas (`ApiCliente.empresa`) sí mantienen su FK 1:1.